Chapitre 1 : Introduction à l'algorithmique

Hassen NAKBI ::: hassen.nakbi@gmail.com

1ere année Licence

18 septembre 2023

Introduction

Algorithmique

- L'algorithmique est une science qui étudie la conception et l'analyse d'algorithmes.
- Elle s'intéresse aux méthodes de résolution de problèmes qui utilisent des règles et techniques facilitant la conception d'une solution.

Introduction

Démarche

Les étapes de résolution d'un problème peuvent être définies comme suit :

1. Analyse du problème : Compréhension de la description du problème.
2. Décomposition d'un problème complexe en sous problèmes simples.
3. Spécification du problème.
4. Élaboration d'un algorithme
5. Traduction de l'algorithme sous forme d'un programme.

Introduction

Spécification du problème

La spécification d'un problème est un document qui décrit le problème à résoudre. Elle doit inclure les éléments suivants :

1. **Données d'entrée** : Les informations qui seront fournies en entrée du problème.
2. **Données de sortie** : Les résultats attendus en sortie du problème.
3. **Traitements** : Une séquence d'opérations, des calculs ou les étapes nécessaires pour effectuer la transformation.

Introduction

Spécification du problème

Énoncé du problème : On voudrait calculer et afficher à l'écran la somme des carrés de trois entiers positifs.

1. **Données d'entrée** : Trois valeurs de type entier.
   Les objets sont des valeurs variables ou constantes.
2. **Données de sortie** : Une seule valeur de type entier.
   Le résultat est variable ou constante.
3. **Traitements** : Calculer le carré de chaque entier, puis ajouter ces carrés à la somme totale.
   Utiliser une boucle pour itérer trois fois, les actions de calcul de carré de chaque entier et l'ajout des carrés à la somme totale.

2 Notion d'algorithme

- Définition
- Structure d'un algorithme
- Propriétés

Définition d'un algorithme

Un algorithme est une séquence ordonnée d'instructions, lorsqu'elle est correctement exécutée, aboutit à un résultat spécifique.

2 Notion d'algorithme

- Définition
- Structure d'un algorithme
- Propriétés

Structure d'algorithme

L'écriture d'un algorithme est souple, qu'elle soit exprimée dans un langage informel ou en pseudo-code, afin d'être compréhensible par l'être humain.

Un algorithme est composé de trois parties essentielles :

1. Partie entête qui permet de spécifier le nom d'algorithme.
2. Partie déclarative contient la liste des objets qui devront être utiliser.
3. Partie instruction représente le corps principal de traitement d'un algorithme.

Modèle d'algorithme

Algorithme NomAlgo
TYPE Nom type
CONST Nom constante
VAR Nom variable
DÉBUT
Les instructions d'un algorithme
FIN

Structure d'un algorithme

Exemple d'algorithme

Algorithme Sommecarré
CONST a = 7, b = 11, c = 9
VAR s : entier
DÉBUT
$s \longleftarrow a \times a + b \times b + c \times c$
Écrire("Somme des carrés ", s)
FIN

2 Notion d'algorithme

- Définition
- Structure d'un algorithme
- Propriétés

Propriétés d'un algorithme

- **Clarté** : Un algorithme doit être exprimé de manière claire et compréhensible par tous ceux qui le lisent.
- **Généralité** : Un algorithme doit être conçu de manière à résoudre un problème de manière générale.
- **Efficacité** : Un algorithme doit être formulé pour résoudre un problème en utilisant moins de mémoire et en effectuant un nombre réduit d'actions.
- **Terminant** : Un algorithme doit se terminer après un nombre fini d'étapes, ce qui évite les boucles infinies.

3 Notion de programme

- Définition
- Langage de programmation
- Structure d'un programme

Définition d'un programme

Un programme est une suite d'instructions en code binaire, destinées à être exécuter par un ordinateur.

3 Notion de programme
- Définition
- Langage de programmation
- Structure d'un programme

Langage de programmation

Le langage C est un langage de programmation développé à partir de 1972 dans les laboratoires Bell, dans le but d'écrire le système d'exploitation Unix. Sa paternité est attribuée à Dennis Ritchie et Ken Thompson.
Il s'agit d'un langage de haut niveau, portable, incontournable, largement utilisé dans le monde, dont la syntaxe a fortement influencé et inspiré d'autres langages de plus haut niveau.

Cycle de développement

Un cycle de développement est réalisé à chaque étape de traduction d'un algorithme.

1. Création d'un code source (fichier de type C) avec un éditeur de texte (EDI).
2. Compilation du code source.
3. Édition des liens du code binaire.
4. Exécution d'un programme.

3 Notion de programme

- Définition
- Langage de programmation
- Structure d'un programme

Structure d'un programme C

Un programme en code source de langage C est composé de deux parties essentielles :

1. **Partie entête** qui contient la liste des noms de fichiers bibliothèque, des sous programmes et les objets globaux d'un programme.
2. **Partie instructions** est définie par une fonction principale nommée "main" qui représente le point d'entrée d'exécution d'un programme.

Structure d'un programme C

```
//Partie entête
#include <stdio.h>
//Partie instructions
int main( ) {
printf("Mon premier programme \n en C ");
return 0;}
```

Le symbole barre oblique inverse suivi par la lettre n est utilisé pour faire un retour à la ligne.

Exemple d'un programme C

```
#include <stdio.h>
#define a  7
#define b  11
#define c  9
short s ;
int main( ) {
s = a*a + b*b + c*c ;
printf("Somme des carrés %hi",s) ;
return 0 ;}
```

4 Notion de données

- Définition
- Caractéristiques
- Mutabilité d'une donnée
- Type d'une donnée
- Définition des données

Définition des données

Les données font référence à toute information ou valeur que l'algorithme utilise, mémorise, ou traite pour résoudre un problème. Les données sont essentielles en algorithmique car elles représentent les entrées, les états intermédiaires et les sorties de l'algorithme

4 Notion de données

- Définition
- Caractéristiques
- Mutabilité d'une donnée
- Type d'une donnée
- Définition des données

Caractéristiques des données

Une donnée est caractérisée par :

1. **Type** : Les données ont un type qui définit le genre d'informations qu'elles représentent. Il détermine comment les données sont stockées en mémoire et quelles opérations peuvent être effectuées sur elles.
2. **Identifiant** : Les données sont souvent identifiées par un nom ou un identifiant unique. Cela permet de se référer à ces données dans l'algorithme.
3. **Mutabilité** : La mutabilité des données indique si elles peuvent être modifiées après leur création. Les données immuables ne peuvent pas être modifiées, tandis que les données mutables peuvent être modifiées.

4 Notion de données

- Définition
- Caractéristiques
- Mutabilité d'une donnée
- Type d'une donnée
- Définition des données

Les constantes et variables

- Une donnée **constante** est une valeur qui ne change pas pendant l'exécution d'un algorithme ou d'un programme.
- Une donnée **variable** est une valeur qui peut être modifier au cours de l'exécution d'un algorithme ou d'un programme.

4 Notion de données

- Définition
- Caractéristiques
- Mutabilité d'une donnée
- Type d'une donnée
- Définition des données

Les types de données

Les variables ont un type de données qui définit le genre d'informations qu'elles peuvent contenir, et elles sont généralement identifiées par un nom significatif.

Les types élémentaires sont les types de données de base qui servent à représenter les informations les plus simples.

Les types élémentaires

1. **Booléen** : Représente une valeur de vérité, soit vrai soit faux, utilisée pour les expressions conditionnelles.
2. **Entier** : C'est l'ensemble des entiers relatifs Z en mathématiques.
3. **Réel** : Représente des entiers avec la virgule pour exprimer une partie fractionnaire.
4. **Caractère** : Représente un seul caractère alphanumérique, tel qu'une lettre, un chiffre, un symbole, ou un code ASCII.

Les types en langage C

1.

Les entiers		
char	entier plus petit	1 octet
short	entier plus court	2 octets
int	entier	4 octets
long long	entier plus long	8 octets

Les entiers positifs sont exprimés avec le mot clé **unsigned** au début de chaque type.

2.

Les réels		
float	réel à simple précision	4 octets
double	réel à double précision	8 octets

3.

Les caractères		
char	un seul caractère entre apostrophes	1 octet

4 Notion de données

- Définition
- Caractéristiques
- Mutabilité d'une donnée
- Type d'une donnée
- Définition des données

Définition des constantes

Syntaxe de définition algorithmique

- **CONST C = valeur**
- CONST A = 4
- CONST X = 3.12, Y = 9

Syntaxe de définition en langage C

- **#define C valeur**
- #define A 4
- **const short A = 4 ;**

Définition des variables

Syntaxe de définition algorithmique

- **VAR v : type**
- VAR a,b : entier
- VAR x : réel

Syntaxe de définition en langage C

- **type v ;**
- short a,b ;
- float x = 3.14 ;

5 Notion d'instruction

- Définition
- Lecture des données
- Affichage des données
- Affectation des variables

Définition d'une instruction

Une instruction est une commande qui décrit une action spécifique à effectuer. Elle peut inclure des opérations arithmétiques, des affectation à des variables, évaluation des expressions logiques, des boucles et des appels à des fonctions.

Les instructions de base

1. La lecture des données au clavier.
2. L'affichage des données à l'écran.
3. L'affectation des variables.

5 Notion d'instruction

- Définition
- Lecture des données
- Affichage des données
- Affectation des variables

Lecture des données au clavier

Syntaxe algorithmique

- **Lire(variable)**
- Lire(a,b)

Syntaxe en langage C

- **#include <stdio.h>**
- **scanf("format",adresse variable) ;**
- short a,b ;
- scanf("%*hi*%*hi*",&a,&b) ;

5 Notion d'instruction

- Définition
- Lecture des données
- Affichage des données
- Affectation des variables

Affichage des données à l'écran

Syntaxe algorithmique

- **Écrire(valeur)**
- Écrire(a)
- Écrire("résultat",b)

Syntaxe en langage C

- **#include <stdio.h>**
- **printf("chaîne format",valeur) ;**
- short a,b ;
- printf("valeur a : %*hi* valeur b : %*hi*",a,b) ;

Les formats types en langage C

Type de données	Format	Représentation des données
char	%c	Un caractère (lettre,chiffre et symbole)
	%d	Petit entier décimal signé
	%o	Petit entier octal
	%x	Petit entier hexadécimal
short	%hi | %hd	Entier court signé
unsigned short	%hu	Entier court positif
int	%i | %d	Entier signé
unsigned int	%u	Entier positif
long long	%ll	Entier long signé
float	%f	Réel à simple précision
double	%lf	Réel à double précision
char [ ]	%s	Chaîne de caractères

5 Notion d'instruction

- Définition
- Lecture des données
- Affichage des données
- Affectation des variables

Affectation des variables

L'instruction d'affectation

L'affectation est une opération qui consiste à modifier la valeur d'une variable, en utilisant le symbole $\longleftarrow$.
Cette valeur peut prendre plusieurs formes, comme le contenu d'une autre variable, valeur constante ou bien une expression à évaluer.

Syntaxe d'affectation

- **VAR a : entier**
- **b : réel**
- **DÉBUT**
- a $\longleftarrow$ 11
- b $\longleftarrow$ 6.34
- **FIN**

- **int main( ) {**
- short a ;
- float b ;
- a = 15 ;
- b = 2.75 ;}

Affectation en langage C

L'affectation composée consiste à combiner un opérateur arithmétique devant l'assignation d'une variable pour simplifier de certaines opérations fréquentes.

Affectation en langage C

Les opérateurs composées : +=, -=, *=, /=

a $\longleftarrow$ a + 2	a += 2 ;
b $\longleftarrow$ b * 3	b *= 3 ;

L'instruction d'affectation

Les opérations d'incrémentation et de décrémentation sont utilisées pour augmenter ou diminuer la valeur d'une variable.

Incrémentation et décrémentation

Algorithmique	**Langage C**
a $\longleftarrow$ a + 1	a++ ;
b $\longleftarrow$ b - 1	b- - ;

Opérateur dans une expression

Dans une expression d'affectation, un opérateur est un symbole ou un signe qui indique l'opération à effectuer entre une ou plusieurs expressions pour calculer la nouvelle valeur d'une variable.

Opérateurs arithmétiques		
	Algorithmique	**Langage C**
Entiers	ABS,e^P,*,/, DIV, MOD,+,-	*,/,%, + , -
Réels	ABS,r^P,*,/, +,-	fabs(r),pow(r,p),*,/, + ,-
Caractère	+ , -	+ , -

Les opérateurs logiques et de comparaison

	Algorithmique	Langage C
Logiques	NON, ET et OU	!, && et ||
Comparaison	$<$, $>$, $\leq$, $\geq$, $=$ et $\neq$	<, >, <=, >=, == et !=

Priorité des opérateurs

La priorité des opérateurs dans une expression détermine l'ordre dans lequel les opérations sont effectuées. Les opérateurs sont évalués en suivant un ensemble de règles de priorité.

1. ( ), [ ]
2. !, ++,- -, -, &
3. *, /, %
4. +, -
5. $<$, $\leq$,$>$,$\geq$
6. == et !=
7. &&
8. ||
9. =,*=,/=, %=

Valeur aléatoire

La valeur d'une variable peut être attribuée d'une façon aléatoire, en utilisant la fonction algorithmique Aléa(N) qui génère un entier naturel compris entre 0 et N-1.
Si l'entier E aléatoire est compris entre les valeurs de a et b, où a est différent de zéro, alors N = (b-a) +1.
Par conséquence, E= a + Aléa(N).

Exemple de valeur aléatoire

Algorithmique

```
VAR x,y : entier
DÉBUT
// x compris entre 0 et 11
x ⟵ aléa(12)
// y compris entre -4 et 15
y ⟵ -4 + aléa(20)
FIN
```

Langage C

```
#include <stdlib.h>
short x, y ;
int main( ) {
// x compris entre 0 et 11
x = rand()%12 ;
// y compris entre -4 et 15
y = -4 + rand()%20 ;
return 0 ;}
```

Les fichiers bibliothèques en C

stdio.h	
printf	Affichage de texte à l'écran
scanf	Lecture de données depuis clavier

math.h	
sqrt(r)	Calcule la racine carrée d'un réel
pow(x,y)	Calcule une puissance, x à la puissance y
sin, cos, tan	Calculent les fonctions trigonométriques.
ceil(r)	Arrondit un réel le plus proche ou supérieur.
floor(r)	Arrondit un réel le plus proche ou inférieur.

stdlib.h	
rand()	Génère un entier aléatoire.
srand(time(NULL))	Initialise le générateur de entiers aléatoires.